

TDMA SCHEDULE PLANNER AND OPTIMIZER

Wireless Protocol Development

Submitted by: Divya S

Program: B.Tech Information Technology

Technologies: Python, NetworkX, Pytest

Repository: <https://github.com/Divya-S-19/TDMA-Scheduler>

1. PROJECT OVERVIEW

This project implements a centralized **TDMA Schedule Planner and Optimizer** for a wireless network.

The system takes static coordinates of wireless radio nodes as input and determines which nodes can transmit during the same TDMA time slot without creating transmission conflicts.

The implementation is divided into two parts:

- **Part 1:** TDMA schedule generation using distance-2 graph coloring.
- **Part 2:** EMANE integration approach and schedule adapter prototype.

Part 1 is the main implemented component. Part 2 demonstrates how the generated schedule can be connected to an EMANE-based TDMA simulation environment.

2. PROBLEM STATEMENT

The objective is to create a centralized software system that calculates a conflict-free TDMA schedule for wireless nodes.

Two nodes must not transmit in the same time slot when:

1. They are directly connected by a wireless link.
2. They are separated by two hops, meaning they share a common neighboring node.

Nodes that are sufficiently far apart can reuse the same time slot.

Therefore, the wireless network is first represented as a graph. A second **conflict graph** is then constructed based on the distance-2 rule.

The scheduling problem becomes a graph-coloring problem where:

- Each node represents a wireless radio.
- Each color represents a TDMA time slot.
- Connected nodes in the conflict graph must receive different colors.

The objective is to produce a valid schedule while using as few slots as possible.

3. INPUT

The scheduler accepts node coordinates in JSON format.

The project uses the following 16-node sample topology:

```
{
  "Node_01": [0, 0],
  "Node_02": [300, 0],
  "Node_03": [600, 0],
  "Node_04": [900, 0],

  "Node_05": [0, 300],
  "Node_06": [300, 300],
  "Node_07": [600, 300],
  "Node_08": [900, 300],

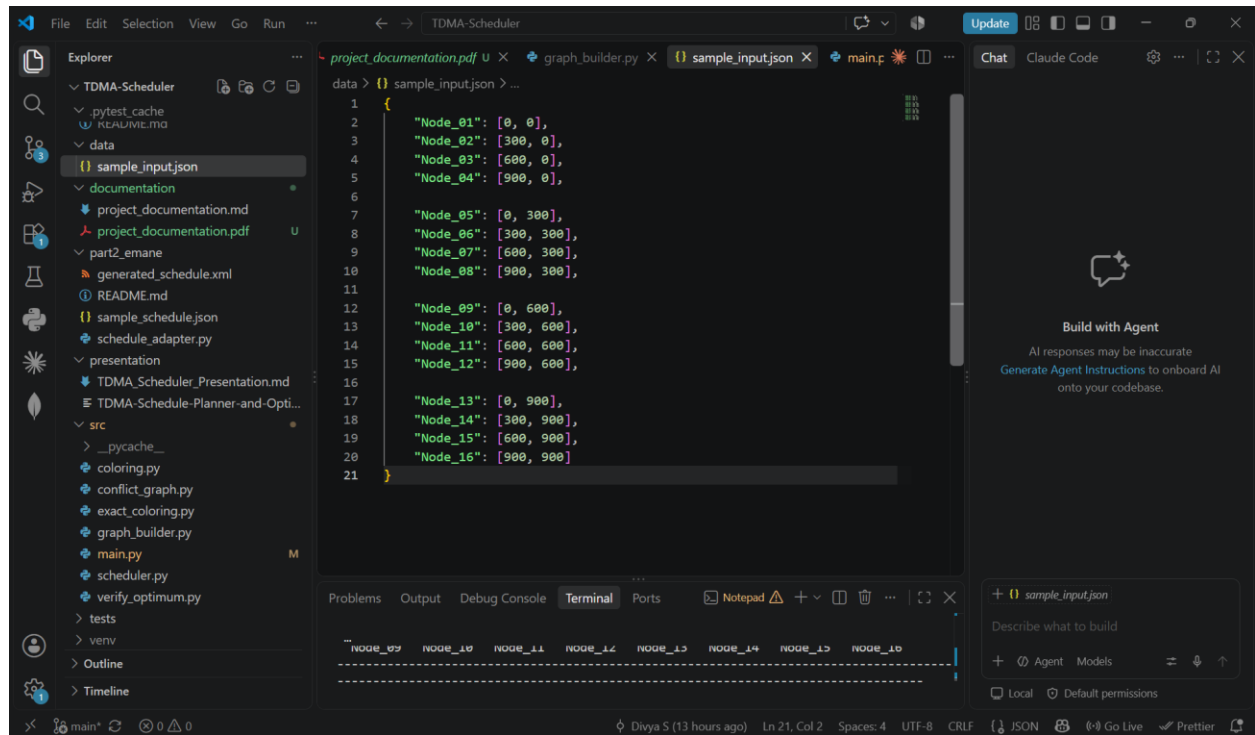
  "Node_09": [0, 600],
  "Node_10": [300, 600],
  "Node_11": [600, 600],
```

```

"Node_12": [900, 600],

"Node_13": [0, 900],
"Node_14": [300, 900],
"Node_15": [600, 900],
"Node_16": [900, 900]
}

```



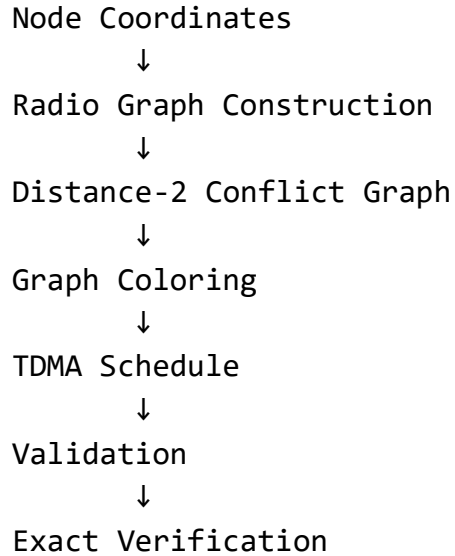
Network Parameters

- Number of nodes: **16**
- Topology: **4 × 4 grid**
- Distance between adjacent grid points: **300 m**
- Communication range: **500 m**
- Coordinates are static.

Two nodes are considered directly connected when their Euclidean distance is less than or equal to 500 m.

4. DESIGN PROCESS

The implementation follows the following pipeline:



Each stage has a specific responsibility.

Stage 1 – Read Coordinates

The scheduler loads the node coordinates from a JSON file.

Stage 2 – Build Radio Graph

A graph is created using NetworkX.

Each node represents a wireless radio.

An edge is added between two nodes when their Euclidean distance is within the 500 m communication range.

Stage 3 – Build Conflict Graph

The radio graph is converted into a distance-2 conflict graph.

Two nodes are connected in the conflict graph if their shortest-path distance in the radio graph is:

1 or 2

This means that both direct neighbors and nodes sharing a common neighbor are prevented from using the same TDMA slot.

Stage 4 – Graph Coloring

The conflict graph is colored using several heuristic ordering strategies.

The candidate strategies include:

- Highest-degree ordering
- Lowest-degree ordering
- DSATUR-style ordering
- NetworkX largest-first ordering

The resulting slot counts are compared and the coloring using the fewest slots is selected.

Stage 5 – Schedule Generation

Each color is mapped to a TDMA slot.

The system generates:

- Node-to-Slot mapping
- Slot-to-Node schedule
- Slot × Node binary/boolean matrix

Stage 6 – Validation

The final schedule is checked against every conflict edge.

If two conflicting nodes have the same slot, the schedule is invalid.

Otherwise, the schedule passes validation.

5. RADIO GRAPH

The first graph represents actual wireless connectivity.

For every pair of nodes, the Euclidean distance is calculated:

$$\text{distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

If:

$$\text{distance} \leq 500 \text{ m}$$

an edge is created.

For the given 16-node topology, the resulting radio graph contains:

42 wireless links.

6. DISTANCE-2 CONFLICT GRAPH

The radio graph alone is not sufficient for scheduling because nodes that are two hops apart must also be prevented from transmitting simultaneously.

For example:

Node A ---- Node B ---- Node C

A and C are not directly connected, but they share Node B as a common neighbor.

Therefore:

A and B → conflict

B and C → conflict

A and C → conflict

The conflict graph is constructed by connecting nodes whose shortest-path distance in the radio graph is at most 2.

For the given topology, the resulting conflict graph contains:

90 conflict pairs.

7. GRAPH COLORING APPROACH

Finding the minimum number of colors for a general graph-coloring problem is computationally difficult.

Therefore, the scheduler uses heuristic coloring strategies rather than relying only on exhaustive search.

The following node-ordering strategies were evaluated:

7.1 Highest Degree

Nodes with more conflict relationships are processed earlier.

7.2 Lowest Degree

Nodes with fewer conflict relationships are processed earlier.

7.3 DSATUR-Style Ordering

Nodes are selected based on their saturation degree and conflict relationships.

7.4 NetworkX Largest-First

NetworkX's largest-first ordering is also evaluated.

For each ordering:

1. Nodes are ordered.
2. Colors are assigned greedily.
3. The number of slots used is calculated.
4. The candidate schedules are compared.
5. The schedule with the smallest slot count is selected.

8. FINAL SCHEDULE

For the given 16-node sample topology, the scheduler produced a valid schedule using:

9 TDMA SLOTS

The final Node-to-Slot mapping is:

Node	Slot
Node_01	8
Node_02	5
Node_03	4
Node_04	8
Node_05	7
Node_06	3
Node_07	2
Node_08	7
Node_09	6
Node_10	1
Node_11	0
Node_12	6
Node_13	8
Node_14	5
Node_15	4
Node_16	8

9. SLOT-WISE SCHEDULE

The resulting transmission schedule is:

TDMA Slot	Transmitting Nodes
0	Node_11
1	Node_10
2	Node_07
3	Node_06
4	Node_03, Node_15

- 5 Node_02, Node_14
- 6 Node_09, Node_12
- 7 Node_05, Node_08
- 8 Node_01, Node_04, Node_13,
Node_16

Nodes appearing in the same slot do not have a distance-2 conflict according to the generated conflict graph.

Scheduler execution and conflict-free 9-slot schedule

```
=====
TDMA RADIO NETWORK
=====
Total nodes : 16
Total links : 42

=====
DISTANCE-2 CONFLICT GRAPH
=====
Total conflict pairs : 90

Coloring optimization: 9 slots selected.

=====
TDMA NODE -> SLOT ASSIGNMENTS
=====
Node_01: Slot 8
Node_02: Slot 5
Node_03: Slot 4
Node_04: Slot 8
Node_05: Slot 7
Node_06: Slot 3
Node_07: Slot 2
Node_08: Slot 7
Node_09: Slot 6
Node_10: Slot 1
Node_11: Slot 0
Node_12: Slot 6
Node_13: Slot 8
Node_14: Slot 5
Node_15: Slot 4
Node_16: Slot 8

Total slots used: 9
```

```

/sample_input.json

Total slots used: 9

=====
TDMA SCHEDULE MATRIX
=====
Slot   Node_01  Node_02  Node_03  Node_04  Node_05  Node_06  Node_07  Node_08
Node_09 Node_10  Node_11  Node_12  Node_13  Node_14  Node_15  Node_16
-----
0      0      0      0      0      0      0      0      0
0      0      0      1      0      0      0      0      0
1      0      0      0      0      0      0      0      0
0      1      0      0      0      0      0      0      0
2      0      0      0      0      0      0      1      0
0      0      0      0      0      0      0      0      0
3      0      0      0      0      0      1      0      0
0      0      0      0      0      0      0      0      0
4      0      0      1      0      0      0      0      0
0      0      0      0      0      0      1      0      0
5      0      1      0      0      0      0      0      0
0      0      0      0      0      1      0      0      0
6      0      0      0      0      0      0      0      0
1      0      0      1      0      0      0      0      0
7      0      0      0      0      1      0      0      1
0      0      0      0      0      0      0      0      0
8      1      0      0      1      0      0      0      0
0      0      0      0      1      0      0      1      0

=====
SCHEDULE VALIDATION
=====
Validation: PASSED
No Distance-2 conflicts found.

```

10. SCHEDULE MATRIX

The schedule can also be represented as a binary Slot × Node matrix.

A value of:

1 → Node is scheduled in that slot

0 → Node is not scheduled in that slot

The matrix represents the same Node-to-Slot mapping in machine-readable form and can be used as the interface for further TDMA processing.

11. VALIDATION

The generated schedule was validated against all conflict relationships.

Validation checks that:

For every conflict pair (u, v):

`slot[u] ≠ slot[v]`

For the final schedule:

- Total nodes: **16**
- Radio links: **42**
- Conflict pairs: **90**
- Slots used: **9**
- Distance-2 conflicts detected: **0**

Validation Result

PASSED

No distance-2 conflicts were found in the final schedule.

12. EXACT COLORING VERIFICATION

Although the main scheduler uses heuristic graph coloring, an exact backtracking coloring implementation was also developed to verify the minimum slot count for the selected sample topology.

The verifier tested whether the conflict graph could be colored using:

1 slot
2 slots
3 slots
...

8 slots

9 slots

The results were:

Number of Slots	Result
1–8	Not possible
9	Possible

Therefore:

9 slots are the minimum required for the given 16-node sample topology.

This exact result applies specifically to the provided sample topology and is not a general claim for all wireless networks.

13. TESTING

Automated tests were created using **Pytest**.

The test suite covers:

1. Radio graph construction.
2. Distance-2 conflict detection.
3. Schedule validity.
4. Schedule matrix generation.

The final test execution produced:

4 passed in 0.39s

Test Result

All 4 tests passed successfully.

14. PROJECT STRUCTURE

The project is organized as follows:

```
TDMA-Scheduler/  
├── src/  
│   ├── main.py  
│   ├── graph_builder.py  
│   ├── conflict_graph.py  
│   ├── coloring.py  
│   ├── scheduler.py  
│   ├── exact_coloring.py  
│   └── verify_optimum.py  
├── data/  
│   └── sample_input.json  
├── tests/  
│   └── test_scheduler.py  
├── part2_emane/  
│   ├── README.md  
│   ├── schedule_adapter.py  
│   ├── sample_schedule.json  
│   └── generated_schedule.xml  
├── documentation/  
│   └── project_documentation.pdf  
├── presentation/  
│   └── TDMA-Schedule-Planner-and-Optimizer.pptx  
├── requirements.txt  
├── README.md  
└── .gitignore
```

15. TECHNOLOGIES USED

Python

Used as the primary programming language for implementing the scheduling system.

NetworkX

Used for:

- Radio graph construction
- Conflict graph construction
- Graph analysis
- Heuristic coloring

Pytest

Used for automated testing of the scheduler components.

JSON

Used for storing the static node coordinates and schedule information.

ReportLab / Documentation Tools

Used to prepare the project documentation as a PDF.

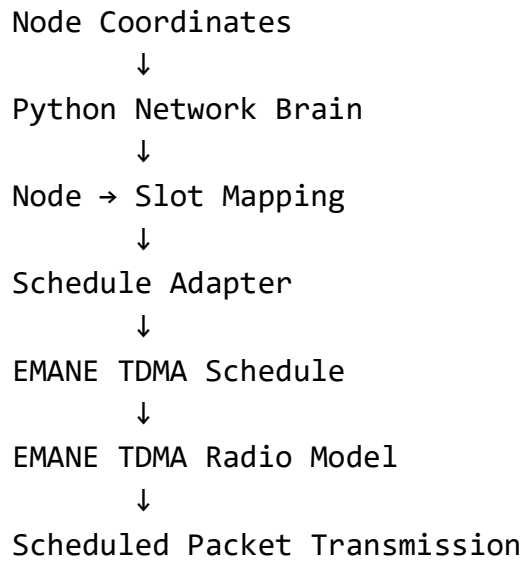
Git and GitHub

Used for version control and project submission.

16. PART 2 – EMANE INTEGRATION APPROACH

The second part of the assignment focuses on integrating the generated TDMA schedule with an EMANE wireless network simulation.

The planned architecture is:



The Python scheduler generates the Node-to-Slot mapping.

A schedule adapter is used to convert this mapping into a structured schedule representation.

The prototype uses a **1 ms slot duration** as specified for the planned TDMA configuration.

17. SCHEDULE ADAPTER PROTOTYPE

A Python schedule adapter has been implemented in:

`part2_emane/schedule_adapter.py`

The adapter:

1. Reads the generated schedule from JSON.
2. Reads each node's assigned slot.
3. Creates a structured XML representation.
4. Stores the generated schedule for the next integration stage.

The prototype output follows the structure:

TDMA Schedule

```
|— Node_01 → Slot 8  
|— Node_02 → Slot 5  
|— Node_03 → Slot 4  
|— ...
```

The generated XML is a **generic prototype representation**.

It is not claimed to be the final EMANE-specific event schema because the exact event format must match the EMANE version and TDMA model configuration used during deployment.

18. CURRENT PART 2 STATUS

Part 2 currently provides the **integration approach and schedule adapter prototype**.

The completed work includes:

- Part 1 schedule generation.
- Node-to-Slot mapping.
- Schedule validation.
- JSON schedule representation.
- Schedule adapter prototype.
- Generic XML schedule generation.
- Planned EMANE integration architecture.

Actual packet-level EMANE execution has **not** been claimed as completed.

The next integration stage would involve configuring the EMANE TDMA Radio Model and connecting the generated schedule to the appropriate EMANE schedule/event mechanism.

19. LIMITATIONS

The current implementation has the following limitations:

1. The scheduler assumes static node coordinates.
2. The communication range is fixed at 500 m.
3. The main scheduler uses heuristic graph coloring.
4. Exact coloring is used as a verification tool for the selected sample topology.
5. The Part 2 XML is a generic adapter prototype and not a guaranteed EMANE event schema.
6. Actual packet-level EMANE execution has not been completed.

20. RESULTS SUMMARY

The implemented system successfully performs the main scheduling workflow.

Metric	Result
Number of nodes	16
Radio links	42
Conflict pairs	90
TDMA slots used	9
Distance-2 conflicts	0
Exact minimum for sample	9
Automated tests	4/4 passed

The final schedule is conflict-free for the given 16-node topology.

21. CONCLUSION

The project implements a centralized TDMA Schedule Planner and Optimizer using graph-based modeling and distance-2 coloring.

The system converts static node coordinates into a radio graph, constructs a distance-2 conflict graph, applies multiple heuristic coloring strategies, and generates a conflict-free TDMA schedule.

For the selected 16-node sample topology, the implementation generated a valid **9-slot schedule**, and exact verification confirmed that fewer than 9 slots are not sufficient for this topology.

Automated testing also passed successfully with **4 out of 4 tests**.

The Part 2 work provides an integration path toward EMANE using a schedule adapter and a planned TDMA model configuration. Further work would connect the generated schedule to the exact EMANE event mechanism and perform packet-level simulation.

22. REPOSITORY

GitHub Repository:

<https://github.com/Divya-S-19/TDMA-Scheduler.git>

The repository contains the source code, test cases, sample input, Part 2 integration prototype, documentation source, and presentation.